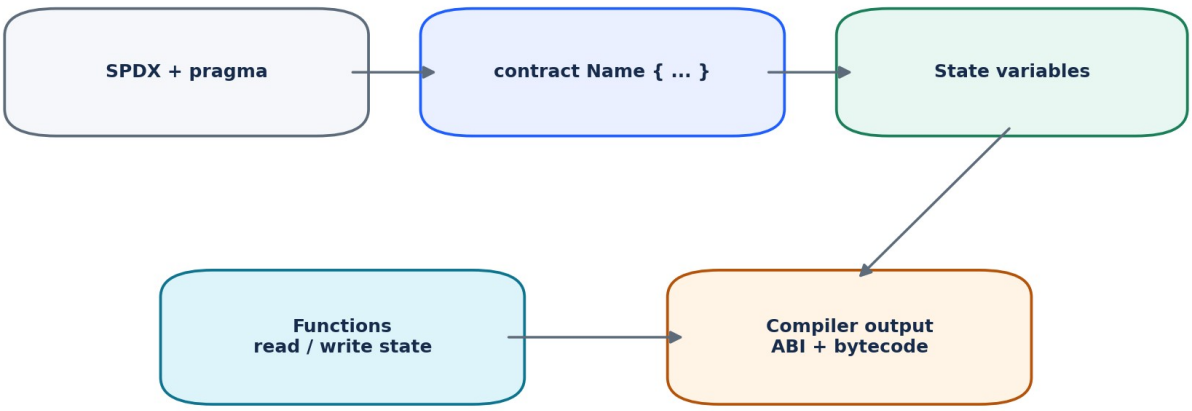
# DAY 5

## Solidity Fundamentals I

## Variables, Types, Functions, Visibility, Data Locations, Arrays & Mappings

**60-MINUTE OUTCOME** Students move from “I know what the EVM is” to “I can read, write, compile, and explain a small Solidity contract.” The focus is syntax plus the mental model behind persistent state and contract calls.




**A Solidity contract combines code (functions) and persistent data (state).**

| Lecture | Duration | Portfolio Outcome |
|---|---|---|
| Day 5 | 60 minutes | A small DeveloperProfileRegistry contract plus code-reading and debugging exercises |
| Level | Beginner | Assumes Days 1-4: blockchain, wallets, EVM, development environment |
| Career focus | LinkedIn + Upwork | Handle basic Solidity creation, edits, bug fixes, and client explanations |

**IMPORTANT** All examples are educational and should be deployed first to Remix VM, Anvil, or a testnet. Do not handle production funds until testing, access control, and security topics are covered.

# 1. Learning Objectives & Minute-by-Minute Schedule

By the end of Day 5, a student should be able to explain what each line of a basic contract does, distinguish persistent state from temporary data, choose basic function visibility and mutability, and use arrays and mappings correctly.

- Read a Solidity source file from SPDX license identifier through the contract body.
- Declare elementary value types: uint/int, bool, address, bytesN, string, and bytes.
- Explain state variables, function parameters, local variables, constants, and immutables.
- Write functions with parameters, return values, visibility, and state mutability.
- Explain public, external, internal, and private without confusing visibility with secrecy.
- Choose storage, memory, and calldata for reference types.
- Create and read dynamic arrays and mappings.
- Build and test a small registry contract in Remix or a local toolchain.

| Minutes | Topic | Instructor Outcome |
|---|---|---|
| 0-5 | What Solidity is + contract anatomy | Connect Day 3 EVM concepts to source code. |
| 5-14 | Variables and elementary types | Students can declare common data types and know defaults. |
| 14-21 | State vs local + constant/immutable | Students understand persistence and storage cost conceptually. |
| 21-33 | Functions, visibility, mutability, returns | Students can design simple read/write functions. |
| 33-42 | storage / memory / calldata | Students stop guessing data locations. |
| 42-49 | Arrays and mappings | Students can model lists and key-value state. |
| 49-57 | Hands-on DeveloperProfileRegistry | Combine concepts in one working contract. |
| 57-60 | Quiz + freelancer task + homework | Check understanding and connect to outsourcing work. |

**TEACHING METHOD** For each keyword, use this sequence: What does it mean? Where does it live? Who can access it? Does it change blockchain state? What client bug happens when it is misunderstood?

# 2. Solidity in the EVM Development Stack | 0-5 min

Solidity is a statically typed, high-level language designed for smart contracts that execute on the Ethereum Virtual Machine. A contract combines code (functions) and data (state) at a blockchain address.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract SimpleStorage {
    uint256 public value;

    function set(uint256 newValue) external {
        value = newValue;
    }

    function get() external view returns (uint256) {
        return value;
    }
}
```

## Read the contract line by line

| Line / Construct | Meaning |
|---|---|
| // SPDX-License-Identifier: MIT | Machine-readable source license identifier. |
| pragma solidity ^0.8.24; | Compiler-version constraint for this source file. |
| contract SimpleStorage | Begins a contract definition. |
| uint256 public value; | Persistent unsigned integer state variable; public generates a getter. |
| function set(...) external | Externally callable write function. |
| function get() external view | Read function that promises not to modify state. |
| returns (uint256) | Declares the function return type. |

**INSTRUCTOR PROMPT** Ask: “When set() changes value, where does the new value live after the transaction finishes?” Correct answer: persistent contract storage/state, not the JavaScript frontend.

## Source code is not what the EVM executes

The compiler produces EVM bytecode for execution and an ABI that tools such as ethers.js use to encode calls and decode results. Day 3 introduced ABI/bytecode; Day 5 now shows how Solidity source produces them.

# 3. Variables and Elementary Types | 5-14 min

Solidity is statically typed: the type of a variable is known at compile time. Type choice affects valid values, operations, ABI encoding, storage layout, and how external applications interact with the contract.

| Type | Example | Use / Beginner Note |
|---|---|---|
| uint256 | uint256 public total = 10; | Unsigned integer. No negative values. uint is an alias for uint256. |
| int256 | int256 public delta = -3; | Signed integer. Can be negative. |
| bool | bool public active = true; | true / false flag. |
| address | address public owner; | 20-byte EVM address. address payable adds native-coin transfer members. |
| bytes32 | bytes32 public id; | Fixed-size 32-byte value; common for hashes/IDs. |
| string | string public name; | UTF-8 text-like dynamic byte sequence; reference type. |
| bytes | bytes public payload; | Dynamic raw byte array; reference type. |

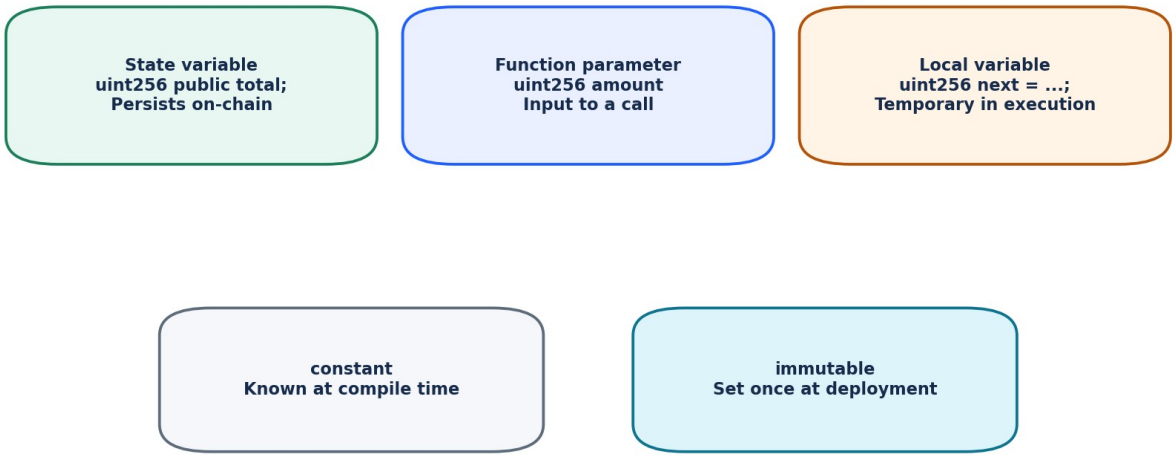
## Default values

| Type | Default |
|---|---|
| uint / int | 0 |
| bool | false |
| address | 0x0000000000000000000000000000000000000000 |
| bytes32 | all zero bytes |
| string / bytes | empty value |
| mapping | conceptually all keys return the value type default until written |

```
contract Defaults {
    uint256 public count;      // 0
    bool public paused;        // false
    address public admin;      // address(0)
    bytes32 public jobId;      // 0x00...00
    string public title;       // ""
}
```

**FREELANCER BUG** A surprising zero value may not mean “the contract failed.” It can simply be the default state because no write has happened yet.

# 4. State, Parameters, Local Variables, constant & immutable | 14-21 min




**Storage writes matter: persistent state is one of the most expensive resources in EVM contracts.**

## State variable

Declared at contract level. Its value is part of contract state and can persist across transactions.

```
uint256 public totalJobs;
```

## Function parameter

Input supplied to a function call. The value exists for that call.

```
function addJobs(uint256 amount) external {
    totalJobs += amount;
}
```

## Local variable

Declared inside a function. It is temporary execution data unless copied into persistent state.

```
function previewTotal(uint256 amount) external view returns (uint256) {
    uint256 nextTotal = totalJobs + amount;
    return nextTotal;
}
```

## constant and immutable

| Keyword | When value is set | Example / Reason |
|---|---|---|
| constant | Compile time | uint256 public constant MAX_FEE_BPS = 1000; |
| immutable | During construction/deployment | address public immutable factory; |
| normal state variable | Can be changed by contract logic | uint256 public totalJobs; |

**GAS MENTAL MODEL** Persistent storage is scarce EVM state. Do not optimize blindly on Day 5, but learn to recognize every line that writes persistent state.

# 5. Operators and Simple Expressions

Solidity supports familiar arithmetic, comparison, boolean, and assignment operators. Since Solidity 0.8.x, ordinary integer arithmetic reverts on overflow/underflow unless placed inside an unchecked block.

| Category | Examples | Meaning |
|---|---|---|
| Arithmetic | + - * / % | Math. Integer division truncates. |
| Comparison | == != < <= > >= | Produces a bool. |
| Boolean | && \|\| ! | AND, OR, NOT. |
| Assignment | = += -= *= | Store/update a value. |
| Ternary | condition ? a : b | Choose one of two values. |

```
function calculate(uint256 price, uint256 quantity)
    external
    pure
    returns (uint256)
{
    return price * quantity;
}
```

## Integer division

```
uint256 x = 5 / 2;  // x == 2, not 2.5
```

**CLIENT PITFALL** Token math is integer math. Prices, percentages, and decimals usually require deliberate scaling (for example token decimals or basis points). Never assume floating-point behavior.

## Mini exercise - predict the result

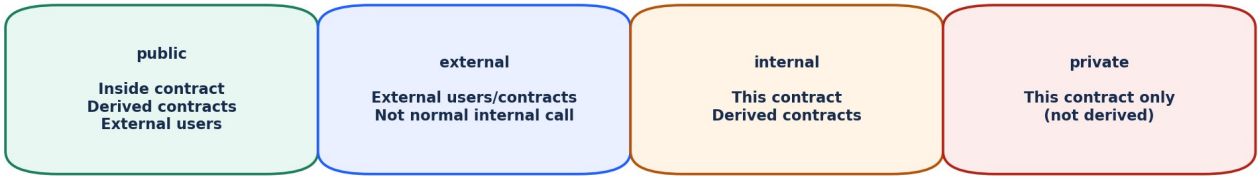
1. If uint256 a = 7; what is a / 3?
2. If bool active = false; what is !active?
3. If count starts at 5 and count += 2 executes, what is count?
4. Can uint256 store -1? Why not?

# 6. Functions: Parameters, Returns, Visibility & Mutability | 21-33 min

```
function functionName(uint256 input)
    external
    view
    returns (uint256 result)
{
    result = input + 1;
}
```

## Function anatomy

| Part | Question to ask |
|---|---|
| Name | What action does this function represent? |
| Parameters | What inputs must the caller provide? |
| Visibility | Who can call it using Solidity access rules? |
| State mutability | Does it read state, modify state, accept native value, or neither? |
| Returns | What data comes back to the caller? |
| Body | What logic executes? |




**Visibility controls Solidity access paths. It does NOT make on-chain data secret.**

Function state mutability is separate: view, pure, payable, or default nonpayable.

## Visibility

| Visibility | Callable from | Key beginner note |
|---|---|---|
| public | Inside + outside; derived contracts | Can be called internally by name and externally. |
| external | Outside via message call | Good for interface-style entry points; not called internally by name. |
| internal | Current + derived contracts | Not exposed as an external ABI entry point. |
| private | Current contract | Not callable from derived contracts. Still NOT secret on a public blockchain. |

**SECURITY FACT** private is an access-control keyword in Solidity source; it is not confidentiality. Blockchain state and

transaction data can be inspected by network participants and explorers.

# 7. Function State Mutability

| Modifier | Can read state? | Can modify state? | Can receive native value? | Typical use |
|---|---|---|---|---|
| view | Yes | No | No | Getters, calculations based on state |
| pure | No | No | No | Math / transformation using only inputs |
| payable | Yes | Yes | Yes | Deposits / functions accepting ETH or native coin |
| none (default nonpayable) | Yes | Yes | No | Ordinary state-changing function |

```
uint256 public score;

function setScore(uint256 newScore) external {
    score = newScore;
}

function getScore() external view returns (uint256) {
    return score;
}

function double(uint256 x) external pure returns (uint256) {
    return x * 2;
}

function deposit() external payable {
    // msg.value contains native currency sent with this call
}
```

## Read call vs write transaction

| Call type | Example | State change? | User normally signs tx? | Gas paid on-chain? |
|---|---|---|---|---|
| Read / eth_call | getScore() | No | No | No transaction fee |
| Write transaction | setScore(100) | Yes | Yes | Yes |
| Payable write | deposit() + value | Usually | Yes | Yes |

**CLIENT COMMUNICATION** When a client says “calling my getter costs gas,” first ask whether they are sending a transaction or performing a read-only RPC call. The same contract function can be invoked differently by tooling.

# 8. Automatic Getters for public State Variables

When a state variable is declared public, the compiler creates an external getter function. This is convenient, but the generated getter follows the shape of the variable and is not always the same as returning an entire complex structure.

```
contract PublicGetterDemo {
    uint256 public count;
    mapping(address => uint256) public points;
}
```

Conceptually, the ABI exposes getter-like calls equivalent to:

```
count() -> uint256
points(address user) -> uint256
```

## Do not create redundant getters without a reason

```
// Redundant for a simple public variable:
function getCount() external view returns (uint256) {
    return count;
}
```

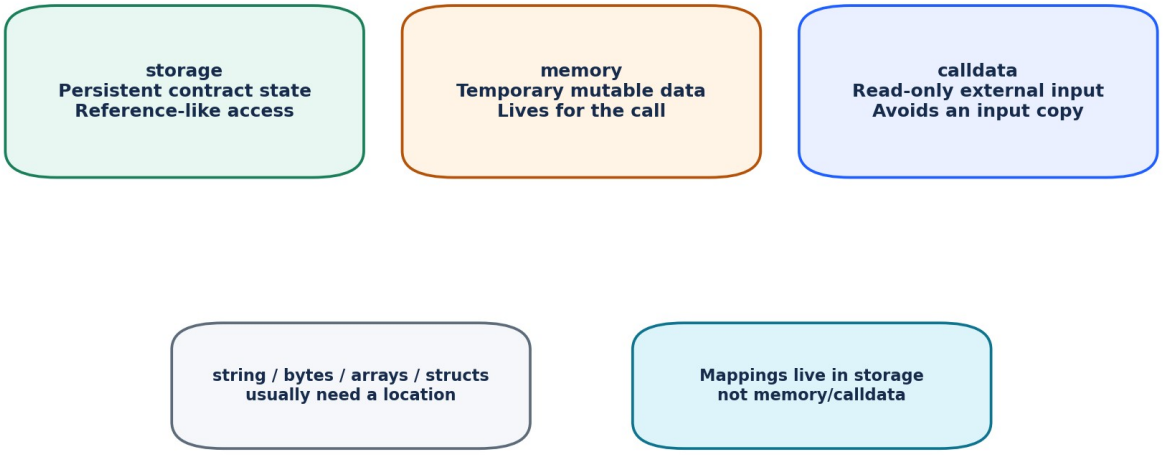
A custom getter can still be valuable when you want validation, combined data, a different return shape, access rules, or an abstraction that can remain stable if storage changes later.

## Mini exercise

Given mapping(address => uint256) public balances;, what input does the generated getter need? Answer: an address key.

**INTERVIEW QUESTION** What does “public” on a state variable do? Strong answer: it allows internal access and causes the compiler to generate an external getter; it does not mean other contracts can directly mutate the variable.

# 9. Data Locations: storage, memory, calldata | 33-42 min




Reference types such as arrays, structs, string, and bytes need a data-location mental model. The location determines lifetime and whether an assignment behaves like a persistent reference or a temporary copy.

| Location | Lifetime / place | Mutable? | Common use |
|---|---|---|---|
| storage | Persistent contract state | Yes (subject to variable mutability) | State variables; internal references to state |
| memory | Temporary for an execution | Yes | Temporary arrays/strings; return data |
| calldata | Read-only call input | No | External function parameters; avoids copying input to memory |

```
string public title;

function setTitle(string calldata newTitle) external {
    title = newTitle;        // calldata input copied into storage
}

function getTitle() external view returns (string memory) {
    return title;            // storage value encoded into return memory
}
```

## Why calldata is common for external inputs

For read-only external parameters, calldata is direct read-only input data. It communicates intent and can avoid a memory copy.

## Storage reference example

```
uint256[] public values;

function replaceFirst(uint256 newValue) external {
    uint256[] storage ref = values;
    ref[0] = newValue;       // modifies persistent values[0]
}
```

**DANGER** A storage reference can modify persistent contract state. A memory copy does not automatically write changes back to storage. This distinction causes many beginner bugs.

# 10. Arrays | 42-46 min

## Fixed-size array

```
uint256[3] public fixedScores;
```

The length is part of the type and cannot change.

## Dynamic array

```
uint256[] public scores;

function addScore(uint256 score) external {
    scores.push(score);
}

function countScores() external view returns (uint256) {
    return scores.length;
}
```

| Operation | Example | Note |
|---|---|---|
| Read | scores[0] | Reverts if index is out of bounds. |
| Append | scores.push(90) | Dynamic storage arrays can grow. |
| Remove last | scores.pop() | Reduces length by one. |
| Length | scores.length | Number of elements. |
| Public getter | scores(0) | Generated getter reads one index, not the entire array. |

**GAS WARNING** Looping over an ever-growing storage array in a state-changing function can eventually become impractical because transaction gas is bounded. “It works with 10 items” is not proof it scales to 100,000.

## Freelancer question

Client asks: “Can I return every user address in one call?” For off-chain reads it may be possible for moderate data, but a production design should consider pagination, events/indexing, storage growth, and RPC response size rather than blindly returning an unbounded list.

# 11. Mappings | 46-49 min

A mapping is key-value storage. It is one of the most common Solidity state structures for balances, permissions, profiles, configuration, and per-user records.

```
mapping(address => uint256) public points;

function setMyPoints(uint256 newPoints) external {
    points[msg.sender] = newPoints;
}

function myPoints() external view returns (uint256) {
    return points[msg.sender];
}
```

| Concept | Explanation |
|---|---|
| Key | The lookup input, e.g. address. |
| Value | The stored result, e.g. uint256. |
| Unset key | Returns the value type default, e.g. 0 for uint256. |
| Enumeration | A mapping does not maintain an iterable list of keys by itself. |
| Location | Mappings are storage-only state structures; they are not standalone memory/calldata values. |

## Mapping + array pattern

```
mapping(address => bool) public registered;
address[] public members;

function register() external {
    if (!registered[msg.sender]) {
        registered[msg.sender] = true;
        members.push(msg.sender);
    }
}
```

The mapping answers “is this address registered?” efficiently; the array preserves an enumerable list of registered addresses.

**DESIGN LESSON** Choose structures based on the questions your application must answer. A mapping is excellent for lookup, but if you need enumeration you often need an additional indexing strategy.

# 12. Global Call Context: msg.sender and msg.value

Solidity exposes globally available values describing the current call. Day 5 only needs two: msg.sender and msg.value.

| Global | Meaning | Example use |
|---|---|---|
| msg.sender | Immediate caller address of the current external call | Store a profile under the caller address; basic access checks later. |
| msg.value | Native currency amount sent with the current call | Deposit/payment functions marked payable. |

```
mapping(address => string) private profiles;

function setMyProfile(string calldata text) external {
    profiles[msg.sender] = text;
}
```

**IMPORTANT** msg.sender is the immediate caller. In multi-contract systems it may be another contract, not the original human wallet. Do not design authentication around assumptions you have not tested.

## One crucial security preview

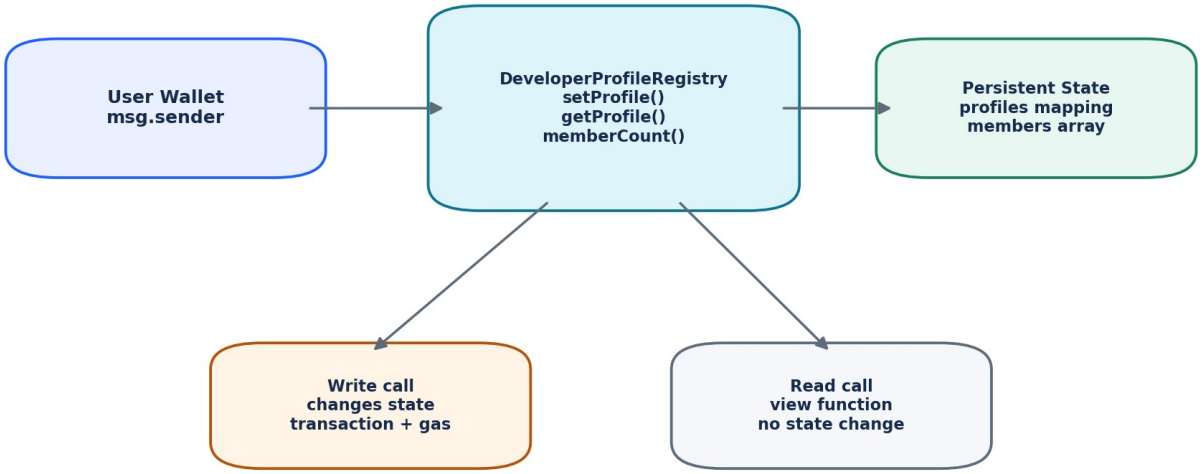
Do not use tx.origin for ordinary authorization. Day 10 will cover authentication and phishing-style call chains in detail. For now, learn msg.sender and avoid copying random authorization snippets from the internet.

## Question for students

If Alice calls Contract A, and Contract A calls Contract B, what is msg.sender inside Contract B? Answer: Contract A.

# 13. Hands-On Build: DeveloperProfileRegistry | 49-57 min

Goal: build a small educational contract where each wallet can publish a short profile string. The contract tracks profiles, whether an address has registered, and a list of member addresses.




```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract DeveloperProfileRegistry {
    mapping(address => string) private profiles;
    mapping(address => bool) public registered;
    address[] public members;

    function setProfile(string calldata newProfile) external {
        if (!registered[msg.sender]) {
            registered[msg.sender] = true;
            members.push(msg.sender);
        }

        profiles[msg.sender] = newProfile;
    }

    function getProfile(address user)
        external
        view
        returns (string memory)
    {
        return profiles[user];
    }

    function memberCount() external view returns (uint256) {
        return members.length;
    }
}
```

**NOT PRODUCTION READY** This is intentionally simple. It has no moderation, length limits, events, upgrade strategy, privacy, anti-spam controls, or economic model. Production requirements belong in later lectures.

# 14. Live Demo Script for the Instructor

## Step 1 - Create the file

In Remix, create DeveloperProfileRegistry.sol and paste the contract. Select a compatible 0.8.x compiler and compile.

## Step 2 - Explain compiler feedback

Show the compile-success indicator. If the compiler reports a syntax/type/data-location error, read the exact line and expected type before changing code.

## Step 3 - Deploy to a local environment

Use Remix VM or a local Anvil environment. Do not use production funds.

## Step 4 - Read initial state

5. Call registered(your account). Expected: false.
6. Call memberCount(). Expected: 0.
7. Call getProfile(your account). Expected: empty string.

## Step 5 - Send a write transaction

Call setProfile("Solidity beginner - Day 5"). Observe that this is a state-changing transaction.

## Step 6 - Read the new state

8. registered(your account) -> true.
9. memberCount() -> 1.
10. members(0) -> your account address.
11. getProfile(your account) -> the stored profile string.

## Step 7 - Update profile again

Call setProfile a second time. memberCount should remain 1 because the mapping prevents a duplicate array insertion.

**DEMO QUESTION** Which exact statements perform persistent storage writes? registered[msg.sender] = true; members.push(msg.sender); profiles[msg.sender] = newProfile;.

# 15. Common Solidity Beginner Errors & How to Debug Them

| Symptom / Error | Likely cause | Debug move |
|---|---|---|
| Parser error near a line | Missing ;, brace, parenthesis, or malformed declaration | Check the error line and the line immediately above it. |
| Data location must be specified | string/bytes/array/struct parameter or return lacks memory/calldata/storage | Ask whether the data is input, temporary output, or persistent state. |
| Function declared view but modifies state | Assignment/push/pop/storage mutation inside view | Remove mutation or redesign function as a state-changing call. |
| Index out of bounds revert | Reading array[i] where i >= length | Inspect length before index access. |
| Mapping key returns 0/false/empty | Key has never been assigned | Remember default values; verify correct key/address/network. |
| Cannot call external function internally by name | external entry point used as if internal | Refactor shared logic into an internal function or revisit visibility. |
| Attempt to modify calldata | calldata is read-only | Copy to memory if mutation is actually required. |
| “private data leaked” | Misunderstood private as encryption | Explain public-chain transparency; never store secrets on-chain. |

## Freelancer debugging sequence

12. Reproduce the compile/runtime problem in a minimal environment.
13. Read the exact compiler or revert message before changing code.
14. Identify the category: syntax, type, visibility, mutability, data location, state, or runtime bounds.
15. Create the smallest test or manual call that proves the fix.
16. Explain the root cause to the client, not only the changed line.

**PRO HABIT** Do not “fix” compiler errors by randomly adding public, memory, payable, or unchecked until compilation passes. Every keyword changes semantics.

# 16. Freelancer / Upwork Client Scenarios

## Scenario A - “Please add a public getter”

First inspect whether the state variable is already public. If it is, a generated getter may already exist. Confirm the desired ABI and whether a custom getter is actually needed.

## Scenario B - “My list does not show all mapping users”

Explain that mappings do not enumerate keys themselves. Propose an array/event/indexer depending on requirements and scale.

## Scenario C - “Make this private so nobody can see it”

Explain that Solidity private does not provide data confidentiality on a public blockchain. Sensitive plaintext should not be stored on-chain merely because a variable is private.

## Scenario D - “This read function asks MetaMask to sign”

Check frontend integration. A view function should normally be invoked as a read call through a provider, not sent as a transaction through a signer.

## Scenario E - “Change string memory to calldata to save gas”

Check function visibility and whether the parameter is only read. calldata is appropriate for read-only external input, but do not mechanically change locations without understanding copy/reference behavior.

| Client request | What you should inspect first |
|---|---|
| Add field to contract | Storage implications, getter/API, initialization/default value |
| Add update function | Who may call it? What state changes? Validation needed? |
| Return all records | Data structure, boundedness, pagination/indexing strategy |
| Reduce gas | Measure first; count storage reads/writes; do not sacrifice correctness/security |
| Fix compile error | Compiler version, exact error, data location, types, imports |

# 17. Interview Questions - Day 5

### 1. What is a state variable?
A contract-level variable whose value is part of persistent contract state.

### 2. uint256 vs int256?
uint256 is unsigned (non-negative); int256 is signed.

### 3. public vs external function?
public supports normal internal calls and external ABI calls; external is an external entry point and is not called internally by name.

### 4. Does private hide blockchain data?
No. It restricts Solidity-level access; public blockchain data is not confidential.

### 5. view vs pure?
view may read state but not modify it; pure may neither read nor modify contract state.

### 6. Why use calldata?
For read-only external reference-type inputs; it communicates immutability and can avoid copying to memory.

### 7. storage vs memory?
storage refers to persistent state; memory is temporary execution data.

### 8. What happens for an unset mapping key?
The mapping returns the default value of its value type.

### 9. Can you iterate all mapping keys automatically?
No. A mapping does not maintain an enumerable key list by itself.

### 10. What does a public state variable provide?
Solidity generates an external getter for it.

# 18. 10-Question Mini Quiz | 57-60 min

17. Which variable persists after a transaction: a state variable or a local variable?
18. What is the default value of bool?
19. Which function modifier means “may read state but may not modify it”?
20. Which modifier lets a function receive native currency?
21. Does private make state secret?
22. Which data location is read-only external input?
23. What does scores.length return?
24. If mapping(address => uint256) points has never stored Alice, what does points[Alice] return?
25. What does msg.sender represent?
26. Why might you pair a mapping with an array?

## Answer key

| # | Answer |
|---|---|
| 1 | State variable. |
| 2 | false. |
| 3 | view. |
| 4 | payable. |
| 5 | No. |
| 6 | calldata. |
| 7 | Number of array elements. |
| 8 | 0. |
| 9 | Immediate caller of the current call. |
| 10 | Mapping for lookup + array for enumeration/order/indexing. |

**PASS STANDARD** A beginner is ready for Day 6 when they can explain at least 8/10 answers and can deploy/call the registry without copying each step blindly.

# 19. Homework & Portfolio Assignment

## Task 1 - Rebuild without copy/paste

Recreate DeveloperProfileRegistry from memory. Compile after every few lines so you learn to isolate mistakes.

## Task 2 - Add one numeric field

Add mapping(address => uint256) public yearsExperience; and a function setYearsExperience(uint256 years) external. Test default and updated values.

## Task 3 - Add a pure helper

Create function double(uint256 x) external pure returns (uint256). Explain why it is pure.

## Task 4 - Write a README

- What the contract does.
- Compiler version / pragma used.
- State variables and why each data structure exists.
- Which functions are read vs write.
- How to deploy in Remix VM.
- Five manual test cases with expected results.
- Known limitations: educational, not production ready.

## Task 5 - Client-style change request

Requirement: “A user should be able to update their profile many times, but the members array must contain that address only once.” Write 3-5 sentences explaining how registered[msg.sender] prevents duplicate member entries.

## Optional challenge

Add function getMember(uint256 index) external view returns (address). Test index 0 and an invalid index. Record what happens and why.

# 20. Day 5 Cheat Sheet

| Concept | Remember |
|---|---|
| Contract | Code + persistent state at an EVM address. |
| uint256 | Unsigned 256-bit integer; common default integer type. |
| address | 20-byte EVM address type. |
| State variable | Persists in contract storage. |
| Local variable | Temporary inside execution. |
| public | Internal + external access; public state variable gets a getter. |
| external | External entry point. |
| internal | Current + derived contracts. |
| private | Current contract only in Solidity; NOT secret. |
| view | May read state; cannot modify it. |
| pure | Does not read or modify contract state. |
| payable | May receive native currency. |
| storage | Persistent state / reference to state. |
| memory | Temporary mutable execution data. |
| calldata | Read-only external input. |
| array | Indexed ordered collection. |
| mapping | Key-value lookup; not automatically enumerable. |
| msg.sender | Immediate caller. |
| msg.value | Native currency sent with call. |

## The 5 questions to ask when reading any Solidity line

27. What type is this data?
28. Where does it live: storage, memory, calldata, stack-like value?
29. Who can access/call it?
30. Does it read or change persistent state?
31. What happens for empty/default/edge-case input?

**NEXT: DAY 6** Solidity Fundamentals II: structs, enums, constructors, modifiers, require/revert/custom errors, events, inheritance, interfaces, receive/fallback, and a client-style escrow/payment contract.

# 21. Official References & Instructor Notes

This lecture follows the official Solidity language documentation. As of August 2026, the Solidity language site lists Solidity v0.8.36 as the current stable release; the documentation “latest” branch may also show development material for the next release. Examples in this lecture use pragma ^0.8.24 so they remain in the Solidity 0.8.x line when compiled with a compatible compiler.

| Reference | What to review |
|---|---|
| https://www.soliditylang.org/ | Current language release and official portal. |
| https://docs.soliditylang.org/en/latest/structure-of-a-contract.html | Contract structure: state variables, functions, modifiers, events, errors, structs/enums. |
| https://docs.soliditylang.org/en/latest/types.html | Value/reference types, arrays, mappings, addresses, data locations. |
| https://docs.soliditylang.org/en/latest/contracts.html | Functions, visibility, state mutability, inheritance-related behavior. |
| https://docs.soliditylang.org/en/latest/introduction-to-smart-contracts.html | Contract/EVM introduction and storage/memory concepts. |
| https://docs.soliditylang.org/en/latest/style-guide.html | Recommended naming and source layout conventions. |

## Instructor emphasis

- Do not let students memorize keywords without a location/state model.
- Repeat that private is not encryption or confidentiality.
- Make students distinguish read calls from state-changing transactions.
- Use the compiler as a teaching tool: type and data-location errors are valuable feedback.
- Keep Day 5 contracts intentionally small. Access control, events, errors, and production patterns arrive on Day 6 and security days.

## Day 5 completion standard

The student can write a basic Solidity contract from a blank file, explain its state variables and functions, choose reasonable visibility/mutability/data locations, use one array and one mapping, compile it, deploy locally, and prove behavior with read/write calls.